

Projet ASD : Structures de Données Hybrides

Seydina Mohamed Diop & Abdou Lahate

23 juillet 2026

1 Introduction

Ce Projet porte sur un systeme de gestion de scolarité . En effet on va concevoir un programme qui va permettre a un utilisateur d enregistrer autant d etudiant qu il souhaite accompagner de leur information personnel et pedagogique .Ainsi une serie de message vont s afficher sur le terminal permettant a l utilisateur d echanger avec la machine .Ce dernier fera des series de calculs liés aux **Structures** et **Fonctions** ci dessous et affichera les informations correspondantes.

2 Modélisation des Structures de Données

Voici les Structures de base que nous avons conçues pour notre **Systeme De Scolarité**.

```
1 typedef struct Note
2 {
3     char matier[20];
4     float valeur;
5     int coef;
6     struct Note *note_suivante; // permet de passer au note
        suivant du meme etudiant
7 }Note;
```

2.0.1 Explication de la structure Note

La structure **Note** reçoit et stocke les résultats académiques d'un étudiant pour chaque matière de la filière, ainsi que le coefficient correspondant. Par exemple, pour un étudiant donné, cette structure va enregistrer ses notes au fur et à mesure. Ensuite, grâce au pointeur `struct Note *note_suivante`, la structure se connecte à la note suivante du même étudiant. C'est ce mécanisme

qui forme une liste chaînée de notes propre à chaque étudiant, permettant de stocker un nombre dynamique de modules sans utiliser de tableau statique.

```

1 typedef struct Etudiant
2 {
3     char nom[50];
4     char prenom[50];
5     int numero_dossier; // identifiant unique par étudiant
6     bool eta_inscription; // 1 si l'étudiant s'est inscrit et
        0 sinon (initialise lors de la creation)
7     float moyenne_general;
8     Note *modules;
9     struct Etudiant *etudiant_suivant; // permet de passer a
        l'étudiant suivant
10 }Etudiant;

```

2.0.2 Explication de la structure Etudiant

La structure `Etudiant` regroupe l'ensemble des informations d'un étudiant de la filière, telles que son nom, son prénom, son numéro de dossier (identifiant unique) et son état d'inscription (via un booleen). Le pointeur `modules` est essentielle : il pointe vers la première note de l'étudiant, liant ainsi l'étudiant à ses résultats. Concernant le pointeur `etudiant_suivant` elle pointe sur l'étudiant suivante creent ainsi une liste chaine

```

1 typedef struct
2 {
3     char nom_filiere[60];
4     int nb_etudiants;
5     Etudiant *premier_du_liste; // point sur le premier de la
        liste des etudiant
6     Etudiant *dernier_du_liste; // point sur le dernier
        etudiant de la liste
7 }Filiere;

```

2.0.3 Explication de la structure Filiere

Cette structure contient le nom d'une filiere donner ainsi que le nombre d'étudiants. Dans la structure `Filiere` on a deux pointeurs : `premier_du_liste` qui pointe sur le premier étudiant de la liste du filière, `dernier_du_liste` quant a lui pointe sur le dernier étudiant. Permettant d'insérer des étudiants directemant a la fin de la liste

3 Modélisation des Fonctions

Voici les fonctions que nous avons conçue pour permettre le bon fonctionnement de notre Systeme de scolarité.

```
1 Etudiant *inscrire_etudiants(Filiere *filiere)
2 {
3     // allocation dynamique d un nouveau etudiant
4     Etudiant *nouveau_etudiant = (Etudiant*) malloc(sizeof(
        Etudiant));
5     if (nouveau_etudiant == NULL)
6     {
7         printf("Ereure : impossible d allouer la memoire");
8         return NULL;
9     }
10    // ajout des information d un etudiant par l utilisateur
11    printf("\n-----Formulaire
        _d_inscription
        -----
        :\n");
12    printf("Donner le nom de l etudiant :\n");
13    scanf("%s", nouveau_etudiant->nom);
14    printf("Donner le prenom de l etudiant :\n");
15    scanf("%s", nouveau_etudiant->prenom);
16    printf("Donner son numero de dossier :\n");
17    scanf("%d", &nouveau_etudiant->numero_dossier);
18
19    nouveau_etudiant->eta_inscription = 1;    // on va
        enregistrer l etat d inscription a 1 par default
20    nouveau_etudiant->modules = NULL ; // pas encore de note
21    nouveau_etudiant->etudiant_suivant = NULL; // se sera le
        dernier etudiant
22
23    if (filiere->premier_du_liste == NULL)
24    {
25        // La liste est vide, il est le tout premier et le
        tout dernier
26        filiere->premier_du_liste = nouveau_etudiant;
27        filiere->dernier_du_liste = nouveau_etudiant;
28    } else
29    {
30        filiere->dernier_du_liste->etudiant_suivant =
            nouveau_etudiant; // le pointeur dernier_du_liste
            pointe sure le nouveau etudiant
31        filiere->dernier_du_liste = nouveau_etudiant; // Il
            devient le nouveau dernier de la filiere
32    }
33
34    filiere->nb_etudiants++;
```

```

35     printf("L'etudiant_%s_%s_a_ete_inscrit_avec_succes!\n",
           nouveau_etudiant->prenom, nouveau_etudiant->nom);
36     return nouveau_etudiant;
37 }

```

3.0.1 Explication de la Fonctions inscrire_etudiant

Cette fonction nous permet d'ajouter de nouveaux etudiant dans une filiere en enregistrant leur information personnel : prenom, nom, numero de dossier grace a l'allocation dynamique suivant : `Etudiant *nouveau_etudiant = (Etudiant*) malloc(sizeof(Etudiant))` qui alloue une espace memoire dans le memoire RAM. Combiner au boucle suivant.

```

1
2     if (filiere->premier_du_liste == NULL)
3     {
4         // La liste est vide, il est le tout premier et le
           tout dernier
5         filiere->premier_du_liste = nouveau_etudiant;
6         filiere->dernier_du_liste = nouveau_etudiant;
7     } else
8     {
9         filiere->dernier_du_liste->etudiant_suivant =
           nouveau_etudiant; // le pointeur dernier_du_liste
           pointe sure le nouveau etudiant
10        filiere->dernier_du_liste = nouveau_etudiant; // Il
           devient le nouveau dernier de la filiere

```

L'utilisateur pourra ajouter autant d'etudiant qu'il souhaite, le programme aura ainsi le nombre totale d'etudiants.

```

1 void note_etudiants(Etudiant *etudiant) // une fonction
   permettant d'enregistrer les resultat academique d'un
   etudiant
2 {
3     if (etudiant == NULL) return;
4
5     // On verifie si l'etudiant est bien inscrit
6     if (etudiant->eta_inscription == 0)
7     {
8         printf("Impossible_d'ajouter_une_note:_cet_etudiant_
           n'est_pas_activement_inscrit.\n");
9         return;
10    }
11    int nouveaux_matiere = 2;
12    while (nouveaux_matiere == 2)
13    {
14        Note *nouveau_note = (Note*)malloc(sizeof(Note)); //
           allocation dynamique

```

```

15     if (nouveau_note == NULL)
16     {
17         printf("Erreur: impossible d'allouer la memoire");
18         return;
19     }
20
21
22     // ajout du resultat d un etudiant
23     printf("\n-----Resultats_\n\n");
24     printf("Donner la matiere: \n");
25     scanf("%s", nouveau_note->matier);
26     do
27     {
28         printf("Donner la note obtenue: \n");
29         scanf("%f", &nouveau_note->valeur);
30         if (nouveau_note->valeur < 0 || nouveau_note->
            valeur > 20)
31         {
32             printf("Erreur! La note doit etre comprise
                entre 0 et 20.\n");
33         }
34     } while (nouveau_note->valeur < 0 || nouveau_note->
        valeur > 20);
35     do
36     {
37         printf("Donner le coefficient du matiere: \n");
38         scanf("%d", &nouveau_note->coef);
39         if (nouveau_note->coef <= 0 || nouveau_note->coef
            > 10)
40         {
41             printf("Erreur! Le coefficient doit etre
                compris entre 1 et 10.\n");
42         }
43     } while (nouveau_note->coef <= 0 || nouveau_note->
        coef > 10);
44
45     nouveau_note->note_suivante = NULL; // c est la
        premiere et dernier note
46
47     if (etudiant->modules == NULL)
48     {
49         // Cas ou l etudiant n a encore aucune note
50         etudiant->modules = nouveau_note;
51     }
52     else
53     {

```

```

54         // Cas ou l'etudiant a deja des notes, on
           parcourt pour aller a la fin
55     Note *note_restante = etudiant->modules;
56     while (note_restante->note_suivante != NULL)
57     {
58         note_restante = note_restante->note_suivante;
59     }
60     // On ajoute la nouvelle note tout a la fin
61     note_restante->note_suivante = nouveau_note;
62 }
63 printf("Matiere_ajouter_avec_succe:\n");
64 printf("Si_vous_vouler_ajouter_une_nouvelle_matiere_
    tape_2_si_non_0:\n");
65 scanf("%d", &nouveaux_matiere);
66 }
67 }

```

3.0.2 Explication de la Fonctions note_etudiant

Comme la fonction precedente celui ci Permet de stocke les resultats scolaire de chaque etudiants enregistrer en amont ,grace a l allocation dynamique suivant `Note *nouveau_note = (Note*)malloc(sizeof(Note))`. Cette fonction va permettre a l utilisateur de donner autant de matiere ainsi que les note et coefficient associer. Pour chacune des etudiants qu il enregistre il peut aussi ne pas ajouter de note pour les etudiants qu il souhaite . Et cela grace au boucle `while (nouveaux_matiere == 2)` qui lui dira de taper 2 s il veut ajouter des matiere et 0 sinon .

```

1 void liste_de_la_classe(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4         // verifie si la filiere est vide ou non
5     {
6         printf("Aucun_etudiant_inscrit_dans_la_filiere.\n");
7         return;
8     }
9     printf("\n-----Liste_des_
    etudiants_de_la_classe
    -----:\n");
10    ;
11    Etudiant *actuel_liste = filiere->premier_du_liste;
12    while (actuel_liste != NULL) // affiche les identifiant
13        des etudiants
14    {
15        printf("Nom:_%s,_Prenom:_%s,_Numero_de_dossier:_%d\n"
16            ,actuel_liste->nom, actuel_liste->prenom,

```

```

        actuel_liste->numero_dossier);
14     actuel_liste = actuel_liste->etudiant_suivant;
15 }
16 }

```

3.0.3 Explication de la Fonctions liste_de_la_classe

Quant a cette fonction il affichera la liste des etudiants que l utilisateur a deja enregistrer .Il donne respectivement leur nom, prenom et numero de dossier.

```

1 void bulletin(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Aucun etudiant inscrit dans la filiere.\n");
6         return;
7     }
8
9     printf("\n-----Bulletin des
    etudiants de la classe
    -----:\n");
10
11     float somme_des_moyennes_classe = 0;
12     int etudiants_avec_notes = 0; // prend en compte que les
    etudiants ayant une moyenne
13
14     Etudiant *actuel_liste_info = filiere->premier_du_liste;
15
16     while (actuel_liste_info != NULL)
17     {
18         // On verifie le statut d inscription de chaque
    etudiant
19         if (actuel_liste_info->eta_inscription == 1)
20         {
21             printf("Nom: %s, Prenom: %s, Numero de dossier: %d\n",
                actuel_liste_info->nom,
                actuel_liste_info->prenom, actuel_liste_info->
                numero_dossier);
22             Note *actuel_liste_bulletin = actuel_liste_info->
                modules;
23             float point_totale = 0;
24             int coef_totale = 0;
25
26             while (actuel_liste_bulletin != NULL)
27             {
28                 printf("Matiere: %s, Note: %f, Coeficiant: %d
                    \n", actuel_liste_bulletin->matier,

```

```

        actuel_liste_bulletin->valeur,
        actuel_liste_bulletin->coef);
29     point_totale += actuel_liste_bulletin->valeur
        * actuel_liste_bulletin->coef;
30     coef_totale += actuel_liste_bulletin->coef;
31     actuel_liste_bulletin = actuel_liste_bulletin
        ->note_suivante;
32 }
33
34 if (coef_totale != 0)
35 {
36     actuel_liste_info->moyenne_general =
        point_totale / coef_totale;
37     printf("Moyenne_Generale: %.2f\n\n",
        actuel_liste_info->moyenne_general);
38     somme_des_moyennes_classe +=
        actuel_liste_info->moyenne_general;
39     etudiants_avec_notes++;
40 }
41 else
42 {
43     actuel_liste_info->moyenne_general = 0;
44     printf("Aucune_note_enregistre\n");
45 }
46 }
47 // On passe a l etudiant suivant
48 actuel_liste_info = actuel_liste_info->
    etudiant_suivant;
49 }
50
51 // Affichage de la moyenne gennerale
52 if (etudiants_avec_notes != 0)
53 {
54     float moyenne_general_classe =
        somme_des_moyennes_classe / etudiants_avec_notes;
55     printf("MOYENNE_GENERALE_DE_LA_FILIERE: %.2f / 20\n"
        , moyenne_general_classe);
56 }
57 else
58 {
59     printf("Les etudiants non pas de note.\n");
60 }
61 }

```

3.0.4 Explication de la Fonctions bulletin

La fonction bulletin permet de calculer les moyenne de chaque etudiant ainsiue la moyenne gennerale de la classe. Elle va aller chercher les module

de chaque étudiants c est a dire ces matiere ainsi que les coef et notes associer pour calculer sa moyenne. Si l etudiant n a pas de note la boucle :

```

1         else
2         {
3             actuel_liste_info->moyenne_general = 0;
4             printf("Aucune note enregistre\n");
5         }

```

Va afficher a l ecran que cette etudiant n a pas de note ,cette etudiant ne sera pas prise en compte pour le calcul de la moyenne gennerale de la classe . Grace au variable `etudiants_avec_notes` , innicialise a 0 qui n avance que si l etudiant qu on doit calculer la moyenne a une note .De cette facon avec la boucle :

```

1     if (etudiants_avec_notes != 0)
2     {
3         float moyenne_general_classe =
4             somme_des_moyennes_classe / etudiants_avec_notes;
5         printf("MOYENNE GENERALE DE LA FILIERE : %.2f / 20\n",
6             , moyenne_general_classe);
7     }
8     else
9     {
10        printf("Les etudiants non pas de note.\n");
11    }
12 }

```

Cqlcule et affiche la moyenne gennerale exacte de la classe.

```

1 void recherche_et_modification_etudiant(Filiere *filiere)
2 {
3     int votre_choix = 3;
4     int choix;
5     char matier[50];
6     float note;
7     int coef;
8     while (votre_choix == 3)
9     {
10        printf("Voulez vous rechercher ou modifier les info d
11            un autre etudiant si oui taper 3 si non 0:\n");
12        scanf("%d", &votre_choix);
13        if (votre_choix == 0)
14        {
15            break;
16        }
17        if (filiere == NULL || filiere->premier_du_liste ==
18            NULL)
19        {
20            printf("Aucun etudiant dans la filiere.\n");
21        }
22    }
23 }

```

```

19         return;
20     }
21     int id_rechercher;
22     printf("Donner le numero de dossier de l'etudiant \n");
23     scanf("%d", &id_rechercher);
24
25     Etudiant *etudiant_actuel = filiere->premier_du_liste
26     ;
27     while (etudiant_actuel != NULL && etudiant_actuel->
28     numero_dossier != id_rechercher)
29     {
30         etudiant_actuel = etudiant_actuel->
31         etudiant_suivant;
32     }
33     if (etudiant_actuel == NULL)
34     {
35         printf("%d ne correspond a aucun etudiant \n", id_rechercher);
36     }
37     else
38     {
39         printf("\n
40         -----
41         Etudiant trouver
42         -----\\n");
43         printf("Nom: %s, Prenom: %s, Numero de dossier: %d, Etat d'inscription: %s", etudiant_actuel->
44         nom, etudiant_actuel->prenom, etudiant_actuel->numero_dossier, (etudiant_actuel->
45         eta_inscription == 1) ? "Inscrit" : "Non inscrit");
46         printf("\n-----Que
47         souhaitez-vous modifier
48         -----\\n");
49         printf("[1] Le Nom\n");
50         printf("[2] Le Prenom\n");
51         printf("[3] Le Numero de dossier\n");
52         printf("[4] L'etat d'inscription (1: inscrit, 0: Suspendu)\n");
53         printf("[5] Matiere Note et Coef\n");
54         printf("Votre choix: ");
55         scanf("%d", &choix);
56     }
57     switch (choix) // cette partie va permettre a l
58     utilisateur de modifier les parametre qu il
59     souhaite

```

```

48     {
49         case 1:
50             printf("Entrez le nouveau nom : \n");
51             scanf("%s", etudiant_actuel->nom);
52             printf("Nom modifie avec succes ! \n");
53             break;
54         case 2:
55             printf("Entrez le nouveau prenom : \n");
56             scanf("%s", etudiant_actuel->prenom);
57             printf("Prenom modifie avec succes ! \n");
58             break;
59         case 3:
60             printf("Entrez le nouveau numero de dossier : \n");
61             scanf("%d", &etudiant_actuel->numero_dossier);
62             printf("Numero de dossier modifie avec succes ! \n");
63             break;
64         case 4:
65             printf("Entrez le nouveau statut (1 pour Inscrit, 0 pour Non inscrit) : \n");
66             scanf("%d", &etudiant_actuel->eta_inscription);
67             printf("Statut d'inscription mis a jour ! \n");
68             break;
69         case 5:
70             if (etudiant_actuel->modules == NULL)
71             {
72                 etudiant_actuel->modules = malloc(sizeof*(etudiant_actuel->modules));
73                 if (etudiant_actuel->modules == NULL)
74                 {
75                     printf("Ereur de l'allocation dynamique");
76                 }
77             }
78             Note *etudiant_actuel_modules = NULL;
79             Note *actuel = etudiant_actuel->modules;
80             while (actuel->note_suivante != NULL)
81             {
82                 actuel = actuel->note_suivante;
83             }
84             etudiant_actuel_modules = actuel;
85
86             printf("Entrez la nouvelle matiere : \n");
87             scanf("%s", matier);
88             if (strcmp(etudiant_actuel_modules->matier,

```

```

matier) == 0 )
89     {
90         printf("Entrez la nouvelle note:\n");
91         scanf("%f", &etudiant_actuel_modules->valeur)
          ;
92         printf("Entrez le nouveau coef:\n");
93         scanf("%d", &etudiant_actuel_modules->coef);
94         printf("La matiere ainsi que sa note et son
          coef on ete modifier avec succe!\n");
95
96     }
97     else
98     {
99         etudiant_actuel_modules->note_suivante =
          malloc(sizeof(Note));
100         if (etudiant_actuel_modules->note_suivante ==
          NULL)
101         {
102             printf("Erreur d allocation memoire\n");
103             break;
104         }
105         strcpy(etudiant_actuel_modules->note_suivante
          ->matier,matier);
106         printf("Donner la note de la nouvelle matier
          :\n");
107         scanf("%f", &note);
108         etudiant_actuel_modules->note_suivante->
          valeur = note;
109         printf("Donner le coef de la nouvelle matier
          :\n");
110         scanf("%d", &coef);
111         etudiant_actuel_modules->note_suivante->coef
          = coef;
112         etudiant_actuel_modules->note_suivante->
          note_suivante = NULL;
113     }
114     break;
115 }
116 }
117 }

```

3.0.5 Explication de la Fonctions recherche_et_modification_etudiant

Cette fonction va demander a l utilisateur s il veut rechercher un etudiant afin de modifier ou non ces information personnel ou academiques en l affichant un message lui permettant de faire deux choix soit taper 3 pour continuer et avoir la possibiliter de modifier certaine information .Soit

taper 0 pour arreter le programme. et eventuellement ne rien modifier ou
sortire apres des modification.

```
1 void supprimer_etudiant(Filiere *filiere)
2 {
3     int votre_choix = 4;
4     int id_supprimer;
5     while (votre_choix == 4)
6     {
7         printf("Si vous voulez supprimer une autre etudiant
8             tape 4 sinon 0:\n");
9         scanf("%d", &votre_choix);
10        if (votre_choix == 0)
11        {
12            break;
13        }
14        if (filiere == NULL || filiere->premier_du_liste ==
15            NULL)
16        {
17            printf("Il n'y a aucune etudiant");
18            return;
19        }
20        printf("Donner le numero de dossier de l etudiant que
21            vous souhaitez supprimer de la filiere:\n");
22        scanf("%d", &id_supprimer);
23        Etudiant *actuel_etudiant = filiere->premier_du_liste
24        ;
25        Etudiant *precedent_etudiant = NULL;
26        while (actuel_etudiant != NULL && id_supprimer !=
27            actuel_etudiant->numero_dossier)
28        {
29            precedent_etudiant = actuel_etudiant; // l
30            etudiant precedent devient l actuel etudiant
31            actuel_etudiant = actuel_etudiant->
32            etudiant_suivant; // l actuel etudiant pas a l
33            etudiant suivant
34        }
35        if (actuel_etudiant == NULL)
36        {
37            printf("Etudiant introuvable avec l id: %d\n",
38                id_supprimer);
39        }
40        else
41        {
42            if (precedent_etudiant == NULL)
43            {
44                // correspond au premier etudiant
```

```

37         filiere->premier_du_liste = actuel_etudiant->
           etudiant_suivant;
38         // cas ou la liste contient q un seul etudiant
39         if (filiere->premier_du_liste == NULL)
40         {
41             filiere->dernier_du_liste = NULL;
42         }
43
44     }
45     else
46     {
47         // 1 etudiant est au milieu ou a la fin
48         precedent_etudiant->etudiant_suivant =
           actuel_etudiant->etudiant_suivant;
49         if (actuel_etudiant == filiere->
           dernier_du_liste)
50         {
51             filiere->dernier_du_liste =
           precedent_etudiant;
52         }
53     }
54     free(actuel_etudiant); // liber l espace alloué
           a l etudiant supprimer
55     filiere->nb_etudiants--;
56     printf("L'etudiant avec l'id: %d a été supprimé
           avec succès\n", id_supprimer);
57 }
58 }
59 }

```

3.0.6 Explication de la Fonctions supprimer_etudiant

De la meme maniere que la fonction precedente celui ci lui permettra de supprimer ou non un etudiant de la liste

```

1 void maximum(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Il n'y a aucune etudiant");
6         return;
7     }
8     float majorant = 0.0;
9     Etudiant *etudiant_major = filiere->premier_du_liste;
10    Etudiant *le_major = NULL; // Pointeur pour memoriser le
           majorant
11    while (etudiant_major != NULL)
12    {

```

```

13         if (etudiant_major->eta_inscription == 1 &&
14             etudiant_major->moyenne_general >= 0.0)
15         {
16             if (etudiant_major->moyenne_general > majorant)
17             {
18                 majorant = etudiant_major->moyenne_general;
19                 // mise a jour du nouveaux majorant
20                 le_major = etudiant_major; // memorise l
21                 // emplacement du majorant actuelle
22             }
23         }
24         etudiant_major = etudiant_major->etudiant_suivant;
25     }
26     if (le_major != NULL)
27     {
28         printf("Le_majorant_de_la_filieres:%s,est%s,%s,
29                numero_de_dossier:%d,et_a_une_moyenne_generale
30                de:%.2f\n",filieres->nom_filieres, le_major->prenom
31                , le_major->nom, le_major->numero_dossier,
32                le_major->moyenne_general);
33     }
34 }

```

3.0.7 Explication de la Fonctions maximum

Cette fonction va chercher parmit tout les etudiant celui qui a la plus grande moyenne et a la fin du programme il donne sont nom prenom numero de dossier sa moyenne et le nom de la filiere.

```

1 void minimum(Filieres *filieres)
2 {
3     if (filieres == NULL || filieres->premier_du_liste == NULL)
4     {
5         printf("Il_n_y_a_aucune_etudiant");
6         return;
7     }
8     float minorant = 20;
9     Etudiant *etudiant_minor = filieres->premier_du_liste;
10    Etudiant *le_minor = NULL; // Pointeur pour memoriser le
11    // minorant
12    while (etudiant_minor != NULL)
13    {
14        if (etudiant_minor->eta_inscription == 1 &&
15            etudiant_minor->moyenne_general >= 0.0)
16        {
17            if (etudiant_minor->moyenne_general < minorant)
18            {
19                minorant = etudiant_minor->moyenne_general;
20                le_minor = etudiant_minor;
21            }
22        }
23        etudiant_minor = etudiant_minor->etudiant_suivant;
24    }
25    if (le_minor != NULL)
26    {
27        printf("Le_moyenne_maximale_de_la_filieres:%s,est:%.2f,
28                le_nom_et_prenom_sont:%s,%s,
29                le_numero_dossier_est:%d\n",
30                filieres->nom_filieres, minorant,
31                le_minor->nom, le_minor->prenom,
32                le_minor->numero_dossier);
33    }
34 }

```

```

17         minorant = etudiant_minor->moyenne_general;
           // mise a jour du nouveaux minorant
18         le_minor = etudiant_minor; // memorise l
           enplacement du minorant actuelle
19     }
20 }
21     etudiant_minor = etudiant_minor->etudiant_suivant;
22 }
23 if (le_minor != NULL)
24 {
25     printf("L'etudiant ayant la plus faible note de la
           filiere: %s, est %s, %s, numero de dossier: %d,
           et a une moyenne generale de: %.2f\n", filiere->
           nom_filiere, le_minor->prenom, le_minor->nom,
           le_minor->numero_dossier, le_minor->
           moyenne_general);
26 }
27 }
28 // tri par ordre de merite (tri par selection)
29 void du_majorant_minorant(Filiere *filiere)
30 {
31     if (filiere == NULL || filiere->premier_du_liste == NULL)
32     {
33         printf("Il n'y a aucune etudiant");
34         return;
35     }
36     Etudiant *i = filiere->premier_du_liste;
37     Etudiant *j;
38     Etudiant *major_minor = NULL; // Pointeur pour memoriser
           le i
39     // il nous faut des variable et une pointeur pour les
           echange fineaux
40     char tmp_nom[50];
41     char tmp_prenom[50];
42     int tmp_id;
43     int tmp_etat_inscription;
44     float tmp_moyenne_general;
45     Note *tmp_modules;
46     while (i != NULL)
47     {
48         if (i->eta_inscription == 1)
49         {
50             major_minor = i; // on suppose que le plus grant
           est le i
51             j = i->etudiant_suivant;
52             while (j != NULL)
53             {
54                 if (j->moyenne_general > major_minor->
                    moyenne_general)

```

```

55         {
56             major_minor = j; // j devient
                               temporairement le plus grand
57         }
58         j = j->etudiant_suivant; // j passe a l
                               etudiant suivant
59     }
60     if (major_minor != i)
61     {
62         strcpy(tmp_nom, i->nom);
63         strcpy(tmp_prenom, i->prenom);
64         tmp_id = i->numero_dossier;
65         tmp_etat_inscription = i->eta_inscription;
66         tmp_moyenne_general = i->moyenne_general;
67         tmp_modules = i->modules;
68
69         strcpy(i->nom, major_minor->nom);
70         strcpy(i->prenom, major_minor->prenom);
71         i->numero_dossier = major_minor->
            numero_dossier;
72         i->eta_inscription = major_minor->
            eta_inscription;
73         i->moyenne_general = major_minor->
            moyenne_general;
74         i->modules = major_minor->modules;
75
76         strcpy(major_minor->nom, tmp_nom);
77         strcpy(major_minor->prenom, tmp_prenom);
78         major_minor->numero_dossier = tmp_id;
79         major_minor->eta_inscription =
            tmp_etat_inscription;
80         major_minor->moyenne_general =
            tmp_moyenne_general;
81         major_minor->modules = tmp_modules;
82     }
83     i = i->etudiant_suivant;
84 }
85 }
86
87
88 }

```

3.0.8 Explication de la Fonctions minimum

Pareil pour la fonction minimum ,il donne l etudiant ayant la plus faible moyenne de la filiere .

```
1 void du_majorant_minorant(Filiere *filiere)
```

```

2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Il n'y a aucune etudiant");
6         return;
7     }
8     Etudiant *i = filiere->premier_du_liste;
9     Etudiant *j;
10    Etudiant *major_minor = NULL; // Pointeur pour memoriser
        le i
11    // il nous faut des variable et une pointeur pour les
        echage fineaux
12    char tmp_nom[50];
13    char tmp_prenom[50];
14    int tmp_id;
15    int tmp_etat_inscription;
16    float tmp_moyenne_general;
17    Note *tmp_modules;
18    while (i != NULL)
19    {
20        if (i->eta_inscription == 1)
21        {
22            major_minor = i; // on suppose que le plus grant
                est le i
23            j = i->etudiant_suivant;
24            while (j != NULL)
25            {
26                if (j->moyenne_general > major_minor->
                    moyenne_general)
27                {
28                    major_minor = j; // j devient
                        temporairement le plus grand
29                }
30                j = j->etudiant_suivant; // j passe a l
                    etudiant suivant
31            }
32            if (major_minor != i)
33            {
34                strcpy(tmp_nom, i->nom);
35                strcpy(tmp_prenom, i->prenom);
36                tmp_id = i->numero_dossier;
37                tmp_etat_inscription = i->eta_inscription;
38                tmp_moyenne_general = i->moyenne_general;
39                tmp_modules = i->modules;
40
41                strcpy(i->nom, major_minor->nom);
42                strcpy(i->prenom, major_minor->prenom);
43                i->numero_dossier = major_minor->
                    numero_dossier;

```

```

44         i->eta_inscription = major_minor->
           eta_inscription;
45         i->moyenne_general = major_minor->
           moyenne_general;
46         i->modules = major_minor->modules;
47
48         strcpy(major_minor->nom, tmp_nom);
49         strcpy(major_minor->prenom, tmp_prenom);
50         major_minor->numero_dossier = tmp_id;
51         major_minor->eta_inscription =
           tmp_eta_inscription;
52         major_minor->moyenne_general =
           tmp_moyenne_general;
53         major_minor->modules = tmp_modules;
54     }
55     i = i->etudiant_suivant;
56 }
57 }
58
59
60 }

```

3.0.9 Explication de la Fonctions du majorant_minorant

Cette fonction Permet d'afficher par ordre de mérite tout les étudiants de la filière du majorant à l'étudiant ayant la plus faible moyenne grâce à la trie par sélection :

```

1     {
2         if (etudiant_minor->eta_inscription == 1 &&
           etudiant_minor->moyenne_general >= 0.0)
3         {
4             if (etudiant_minor->moyenne_general < minorant)
5             {
6                 minorant = etudiant_minor->moyenne_general;
6                 // mise a jour du nouveaux minorant
7                 le_minor = etudiant_minor; // memorise l
6                 enplacement du minorant actuelle
8             }
9         }
10        etudiant_minor = etudiant_minor->etudiant_suivant;
11    }
12    if (le_minor != NULL)
13    {
14        printf("L'etudiant ayant la plus faible note de la
           filiere: %s, est %s, %s, numero de dossier: %d,
           et une moyenne generale de: %.2f\n", filiere->
           nom_filiere, le_minor->prenom, le_minor->nom,

```

```

        le_minor->numero_dossier, le_minor->
        moyenne_general);
15     }
16 }
17 // tri par ordre de merite (tri par selection)
18 void du_majorant_minorant(Filiere *filiere)
19 {
20     if (filiere == NULL || filiere->premier_du_liste == NULL)
21     {
22         printf("Il n'y a aucune etudiant\n");
23         return;
24     }
25     Etudiant *i = filiere->premier_du_liste;
26     Etudiant *j;
27     Etudiant *major_minor = NULL; // Pointeur pour memoriser
        le i
28     // il nous faut des variable et une pointeur pour les
        echange fineaux
29     char tmp_nom[50];
30     char tmp_prenom[50];
31     int tmp_id;
32     int tmp_etat_inscription;
33     float tmp_moyenne_general;
34     Note *tmp_modules;
35     while (i != NULL)
36     {
37         if (i->eta_inscription == 1)

1 void orde_alphabet(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Il n'y a aucune etudiant\n");
6         return;
7     }
8
9     Etudiant *i = filiere->premier_du_liste;
10    Etudiant *j;
11
12    char tmp_nom[50];
13    char tmp_prenom[50];
14    int tmp_id;
15    int tmp_etat_inscription;
16    float tmp_moyenne_general;
17    Note *tmp_modules;
18
19    while (i != NULL)
20    {
21        if (i->eta_inscription == 1)

```

```

22     {
23         j = i->etudiant_suivant;
24
25         while (j != NULL)
26         {
27             if (strcmp(i->nom, j->nom) > 0)
28             {
29                 strcpy(tmp_nom, i->nom);
30                 strcpy(tmp_prenom, i->prenom);
31                 tmp_id = i->numero_dossier;
32                 tmp_etat_inscription = i->eta_inscription
33                 ;
34                 tmp_moyenne_general = i->moyenne_general;
35                 tmp_modules = i->modules;
36
37                 strcpy(i->nom, j->nom);
38                 strcpy(i->prenom, j->prenom);
39                 i->numero_dossier = j->numero_dossier;
40                 i->eta_inscription = j->eta_inscription;
41                 i->moyenne_general = j->moyenne_general;
42                 i->modules = j->modules;
43
44                 strcpy(j->nom, tmp_nom);
45                 strcpy(j->prenom, tmp_prenom);
46                 j->numero_dossier = tmp_id;
47                 j->eta_inscription = tmp_etat_inscription
48                 ;
49                 j->moyenne_general = tmp_moyenne_general;
50                 j->modules = tmp_modules;
51             }
52             j = j->etudiant_suivant; // j avance
53         }
54         i = i->etudiant_suivant; // i avance
55     }
56
57
58 }

```

3.0.10 Explication de la Fonctions orde_alphabet

Conservant cette fonction elle utilise une **trie a bulle**, et elle va permettre l'affichage de la liste par ordre alphabetique :

```

1         strcpy(i->nom, major_minor->nom);
2         strcpy(i->prenom, major_minor->prenom);
3         i->numero_dossier = major_minor->
           numero_dossier;

```

```

4         i->eta_inscription = major_minor->
           eta_inscription;
5         i->moyenne_general = major_minor->
           moyenne_general;
6         i->modules = major_minor->modules;
7
8         strcpy(major_minor->nom, tmp_nom);
9         strcpy(major_minor->prenom, tmp_prenom);
10        major_minor->numero_dossier = tmp_id;
11        major_minor->eta_inscription =
           tmp_eta_inscription;
12        major_minor->moyenne_general =
           tmp_moyenne_general;
13        major_minor->modules = tmp_modules;
14    }
15    i = i->etudiant_suivant;
16 }
17 }
18
19
20 }
21 // tri par ordre alphabetique (tri a bulle)
22 void orde_alphabet(Filiere *filiere)
23 {
24     if (filiere == NULL || filiere->premier_du_liste == NULL)
25     {
26         printf("Il n'y a aucune etudiant\n");
27         return;
28     }
29
30     Etudiant *i = filiere->premier_du_liste;
31     Etudiant *j;
32
33     char tmp_nom[50];
34     char tmp_prenom[50];

```

4 Modélisation du corps de notre projet : (int main)

C'est ici que nous faisons les appels des différentes fonctions pour donner vie à notre programme.

5 Etude de Complexite

Nous allons etudier la complexite de toutes les fonctions dans le meilleur et pire des cas. Soit n : le nombre d'etudiant inscrit, et m : le nombre de module de chaque etudiant.

5.0.1 Complexite du Fonctions inscrire_etudiant

Cette fonction ne contient que des operations elementaires. Donc par sommation des complexites de chaque ligne de code on obtient un $O(1)$.

5.0.2 Complexite du Fonctions note_etudiant

Dans cette fonction nous avons deux boucles de controle de saisie qui ne dependent ni de n ni de m . Mais du nombre d'erreur de l'utilisateur donc ce sont des $O(1)$. Ce pendant nous avons une troisieme boucle imbriquee : `while (note_restante->note_suivante != NULL)`, ce dernier parcourt toutes les notes d'un etudiant jusqu'a la fin pour y ajouter la nouvelle note donc la boucle tourne m fois. La boucle principale `while (nouveaux_matiere == 2)` s'execute aussi m fois car la fin de la boucle depend du nombre de matiere saisie par l'utilisateur. De ce fait nous avons : Dans le meilleur des cas $O(1)$. Dans le pire des cas $O(m^2)$.

5.0.3 Complexite du Fonctions liste_de_la_classe

Cette fonction d'affichage ne comporte qu'une seule boucle `while (actuel_liste != NULL)`. Cette boucle parcourt tous les etudiants enregistres dans la filiere donc elle tourne n fois, donc nous aurons : Dans le meilleur des cas : $O(1)$. Dans le pire des cas : $O(n)$.

5.0.4 Complexite du Fonctions bulletin

Dans cette fonction nous avons une boucle imbriquee : `while (actuel_liste_bulletin != NULL)`, cette boucle parcourt l'ensemble des modules de chaque etudiant, (m fois). Une boucle principale : `while (actuel_liste_info != NULL)` qui parcourt l'ensemble des etudiants (n fois). Ainsi nous avons : Dans le meilleur des cas : $O(1)$. Dans le pire des cas : $O(nm)$.

5.0.5 Complexite du Fonctions recherche_et_modification_etudiant

La fonction `recherche_et_modification_etudiant` comporte deux boucles imbriquees : `while (etudiant_actuel != NULL && etudiant_actuel->numero_dossier`

`!= id_rechercher`) qui s'exécute tant que l'étudiant recherché n'est pas trouvé et la liste n'est pas finie. `while (actuel->note_suivante != NULL)` qui parcourt toute la liste des notes d'un étudiant. Et une boucle principale `while (votre_choix == 3)` qui ne dépend pas du nombre d'étudiants mais de l'utilisateur donc c'est un $O(1)$. Donc on a : Dans le meilleur des cas : $O(1)$. Dans le pire des cas : $O(n) + O(m)$.

5.0.6 Complexité des Fonctions `supprimer_etudiant`

Dans cette fonction on a une boucle principale `while (votre_choix == 4)` qui ne dépend pas du nombre d'étudiants mais de l'utilisateur donc c'est un $O(1)$. Et une boucle interne `while (actuel_etudiant != NULL && id_supprimer != actuel_etudiant->numero_dossier)` qui parcourt les étudiants en comparant leur id. Donc on a : Dans le meilleur des cas : $O(1)$. Dans le pire des cas : $O(n)$.

5.0.7 Complexité des Fonctions `maximum`

Cette fonction comporte qu'une seule boucle : `while (etudiant_major != NULL)` qui parcourt l'ensemble des étudiants. Donc nous aurons : Dans le meilleur des cas : $O(n)$. Dans le pire des cas : $O(n)$.

5.0.8 Complexité des Fonctions `minimum`

Comme la fonction précédente celui-ci présente aussi une seule boucle `while (etudiant_minor != NULL)` parcourt aussi l'ensemble des étudiants. Ainsi : Dans le meilleur des cas : $O(n)$. Dans le pire des cas : $O(n)$.

5.0.9 Complexité des Fonctions `du_majorant_minorant`

Cette fonction utilise le tri par sélection donc elle comporte une boucle principale et une boucle interne. Chaque une de ces boucles parcourt les étudiants. Donc on a : Dans le meilleur des cas : $O(n)$. Dans le pire des cas : $O(n^2)$.

5.0.10 Complexité des Fonctions `ordre_alphabet`

Pareil que la fonction précédente celui-ci aussi utilise une trie (trie à bulle). De ce fait elle comporte deux boucles : une externe et une interne, chaque boucle parcourt aussi l'ensemble des étudiants. Donc on a : Dans le meilleur des cas : $O(n)$. Dans le pire des cas : $O(n^2)$.

5.0.11 Etude du temps d execution

Nombre d'étudiants (n)	Temps Tri par selection (s)	Temps Tri a bulle (s)
100	0,000000	0,000000
500	0,000000	0,000000
1 000	0,001000	0,011000
1 500	0,004000	0,017000
10 000	0,750000	1,073000
15 000	0,344000	1,577000
50 000	2,977000	24,124001
80 000	8,488000	71,599998
100 000	16,330000	117,851997

TABLE 1 – Comparaison des temps d execution des tris

5.0.12 Graphique

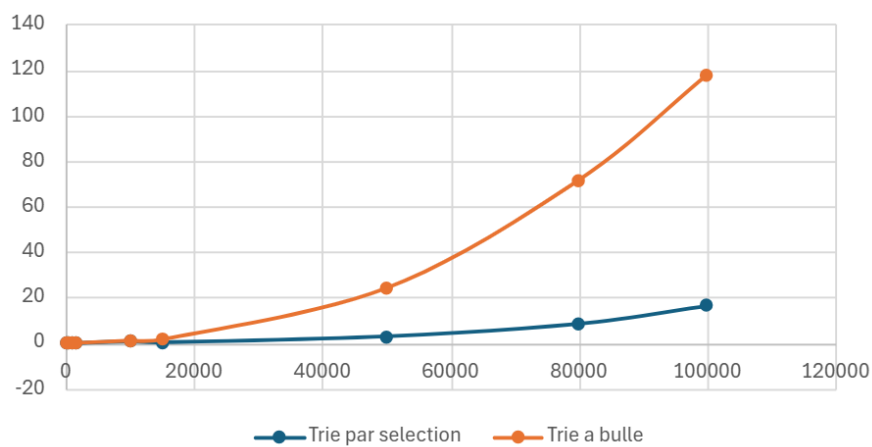


FIGURE 1 – Comparaison du temps d execution entre le tri par selection et le tri a bulle